

Lista de Código: MongoDB com Mongoose

Turma: LionsDev

Tópicos: `mongoose.connect()`, `Schema` (com `type` e `required`), `mongoose.model()`, operações `create` / `find` / `findById` / `findByIdAndUpdate` / `findByIdAndDelete` e `async/await`.

Nesta lista você escreve os schemas e as queries de uma API com banco. Toda operação no banco é assíncrona, então use `async/await` dentro de `try/catch`.

Parte 0: Exercícios de fixação

Itens curtos para praticar a conexão, o schema e as consultas básicas, nesta ordem.

1. Importe o mongoose (`import mongoose from "mongoose"`).
2. Conecte ao banco com `await mongoose.connect(process.env.MONGO_URI)`, lendo a URI do arquivo `.env`.
3. Crie um `Schema` com um campo `nome` do tipo `String`.
4. Torne o campo `nome` obrigatório (`required: true`).
5. Adicione um campo `preco` do tipo `Number` obrigatório.
6. Adicione um campo `ativo` do tipo `Boolean`.
7. Crie o model: `const Produto = mongoose.model("Produto", ProdutoSchema)`.
8. Crie um documento com `await Produto.create({ ... })`.
9. Liste todos os documentos com `await Produto.find()`.
10. Busque um documento pelo id com `await Produto.findById(id)`.

Parte 1: Complete o código

1. Definindo o Schema

Complete o schema com os tipos e validações.

```
const UsuarioSchema = new mongoose.Schema({
  nome: { type: String, required: true },
  email: { /* TODO: String, obrigatório */ },
  idade: { /* TODO: Number, opcional */ },
});
```

```
const Usuario = mongoose.model("Usuario", UsuarioSchema);
```

2. Create com async/await

Complete a função de criação.

```
async function criarUsuario(dados) {  
  try {  
    const novo = /* TODO: await ... crie o documento com 'dados' */;  
    return novo;  
  } catch (erro) {  
    console.log("Erro ao criar:", erro.message);  
  }  
}
```

3. Read por ID

Complete a busca por id.

```
async function buscarUsuario(id) {  
  const usuario = /* TODO: await ... busque por id */;  
  if (!usuario) return "Usuário não encontrado";  
  return usuario;  
}
```

Parte 2: Ache o bug

4. Faltou esperar

Esta função imprime um objeto **Query** do Mongoose em vez da lista de produtos. O que falta?

```
async function listar() {  
  const produtos = Produto.find(); // BUG  
  console.log(produtos);  
}
```

5. Schema sem validação

O campo `preco` deveria ser obrigatório e numérico, mas aceita texto e vazio. Conserte a definição.

```
const ItemSchema = new mongoose.Schema({
  nome: { type: String, required: true },
  preco: String, // BUG: tipo errado e sem required
});
```

Parte 3: Prever o comportamento

6. O que retorna?

Para cada chamada, diga o que a variável recebe (um documento, um array, `null` ou erro).

```
(a) await Produto.find()           -> ?
(b) await Produto.findById("id_que_existe") -> ?
(c) await Produto.findById("id_valido_que_nao_existe") -> ?
(d) await Produto.create({}) // schema exige 'nome' required -> ?
```

Parte 4: Escreva do zero

7. CRUD com Mongoose (Desafio)

Monte o CRUD completo de uma coleção `Livro` com o schema: `titulo` (String, obrigatório), `autor` (String, obrigatório), `ano` (Number) e `disponivel` (Boolean, padrão `true`). Escreva as funções assíncronas, todas com `try/catch`:

- `criarLivro(dados)` → `Livro.create(...)`
- `listarLivros()` → `Livro.find()`
- `buscarLivro(id)` → `Livro.findById(...)` (trate o "não encontrado")
- `atualizarLivro(id, dados)` → `Livro.findByIdAndUpdate(...)` (retorne o atualizado)
- `deletarLivro(id)` → `Livro.findByIdAndDelete(...)`

Depois, conecte ao banco e teste chamando as funções em sequência.

Dica: toda operação de banco é assíncrona. Sem `await`, você recebe uma `Promise` (ou, nas consultas como `find`, um objeto `Query`) no lugar dos dados. Use `try/catch` para capturar erros de validação do schema. Para atualizar e receber o documento já atualizado, passe `{ new: true }` no `findByIdAndUpdate`.

LionsDev • Professor Nicolas Cardoso Motta

Lista de Código: MongoDB com Mongoose - Módulo 08